

What changed in text-to-speech (and speech-to-text), and why

Each change below comes with the defect it fixes, a diagram of the mechanism, and the measurement or technical reason behind it. Every number was measured on this repository with the real Kokoro model, a live server and real headless Chromium, unless it is marked as illustrative. The full audit (best practices, sources, alternatives) is in `docs/voice-stt-tts-audit.md`.

-36%

time to first audio
2.63 s → 1.69 s (6-reply average, 0.65x real-time inference)

-83%

silence between chunks
2.38 s → 0.40 s per reply; worst gap
4.75 s → 1.61 s

0 ms

gap between chunks once synthesized:
scheduled on the exact end sample
(measured in Chromium)

35 s

upper bound on any stuck TTS request.
It used to be unbounded, which froze all later speech

413 → 200

Brave speech-to-text fallback: clips were always over the 12 s server limit, and are now ≤ 11.5 s and end when the user stops talking

+71

new tests (server 79 → 103, client 228 → 275). Lint and build are clean.

Contents

01	The pipeline after the change: what is new and what moved	02	Chunk sizing: the real cause of the pauses
03	Gapless playback on one AudioContext	04	Prefetch depth 2
05	Cancellation, timeouts, retries, watchdogs	06	Bounded server queue and backpressure
07	Threads, warm-up, health and logs	08	Speakable text: emoji, URLs, abbreviations, prompt
09	Languages Kokoro can't speak	10	Speech-to-text: the Brave fallback
11	Files, tests, validation, rollback, limitations		

Machine: Apple M5, Node 24, sherpa-onnx-node 1.13.4, Kokoro v1.0 fp32, voice af_heart, speed 0.95. The machine was shared with other work, so the benchmark takes the best of 3 runs and fits a model, and every strategy is compared under the same load. Nothing was judged by ear.

The pipeline after the change

The architecture is the same: Kokoro runs in-process through sherpa-onnx, and replies stream over SSE. The changes are all at the points where text is sized, requests are bounded and cancelled, and audio is scheduled.

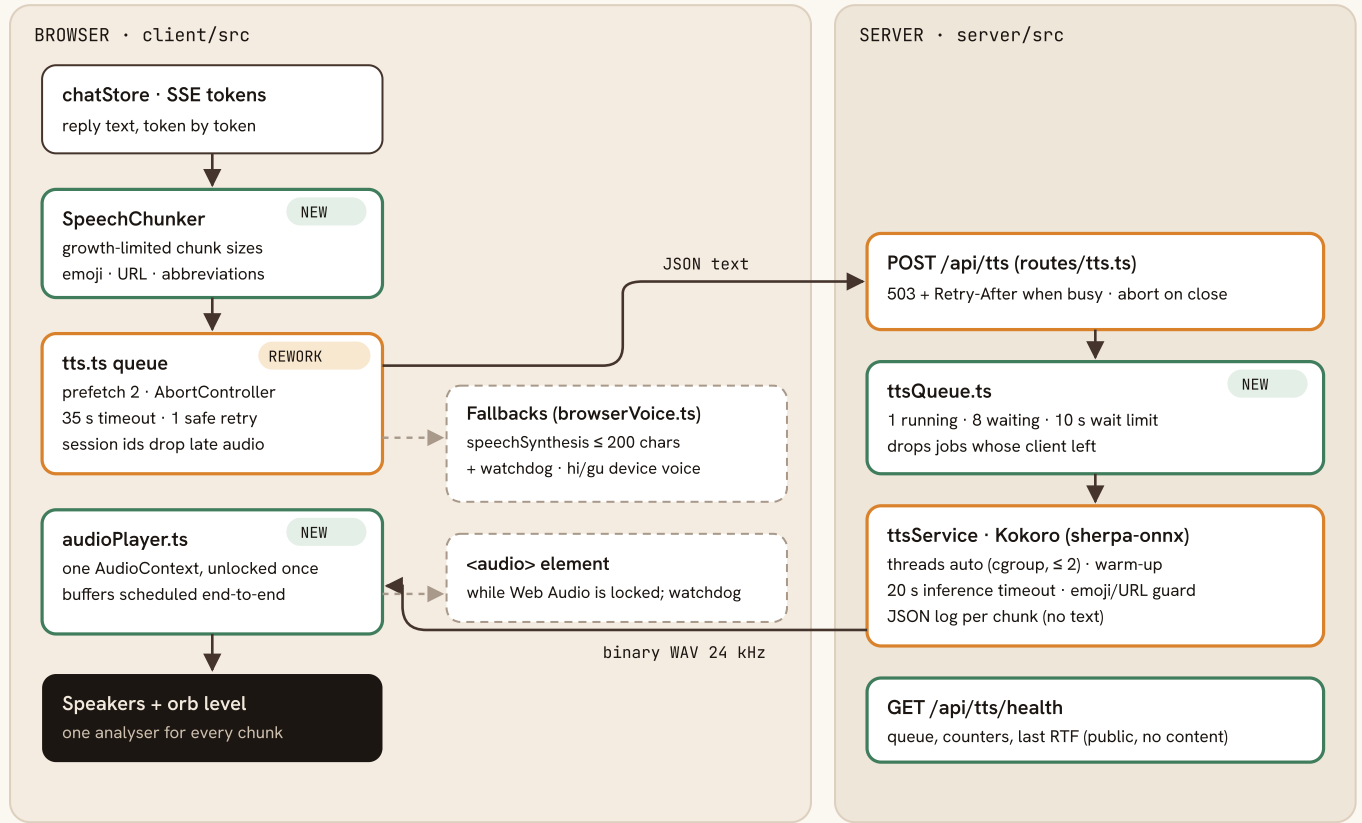


Figure 1. Green = new module, amber = reworked, dashed = fallback. `speech.ts` still exports the same TTS API (it re-exports `tts.ts`), so no caller changed.

The seven reported problems and where they came from

SYMPTOM	ROOT CAUSE FOUND	FIXED BY
Speech freezes or never finishes	TTS fetch had no timeout or abort; stuck <code>ended</code> / <code>onend</code> had no watchdog; Chrome speechSynthesis stops after ~15 s without <code>onend</code>	05
Gaps between chunks	Tiny first chunk followed by a 2-sentence chunk; one <code><audio></code> element started per <code>ended</code> event; prefetch depth 1	02 · 03 · 04
Robotic or flat	Emoji names and URLs read aloud; sentences split at "Dr." / "e.g.", which resets intonation	08
TTS blocks the app	Not the event loop (≤ 27 ms lag measured). The problem was CPU and memory: ~600 MB RSS and an unbounded queue	06 · 07
Responses overlap	Playback was already serialized; stale requests still ran on the server and delayed the next reply	05 · 06
Long replies start slowly	A long first sentence was synthesized whole (7.7 s to first audio on the long reply)	02
Old audio after Stop or a new message	Client discarded late audio, but requests weren't aborted and the server didn't drop queued jobs	05 · 06

Chunk sizing: the real cause of the pauses

NEW

The server synthesizes chunks one after another at about 0.6–0.65× real time. Chunk N+1 is ready in time only if its synthesis fits inside chunk N's playback. The old rule sent one sentence, then two at a time, which gives exactly the wrong shape: a 0.7 s "Oh no!" followed by a 19-word pair that takes about 3 s to synthesize.

"Oh no! That sounds really frustrating. Do you want to vent about it, or would a distraction help more right now?"

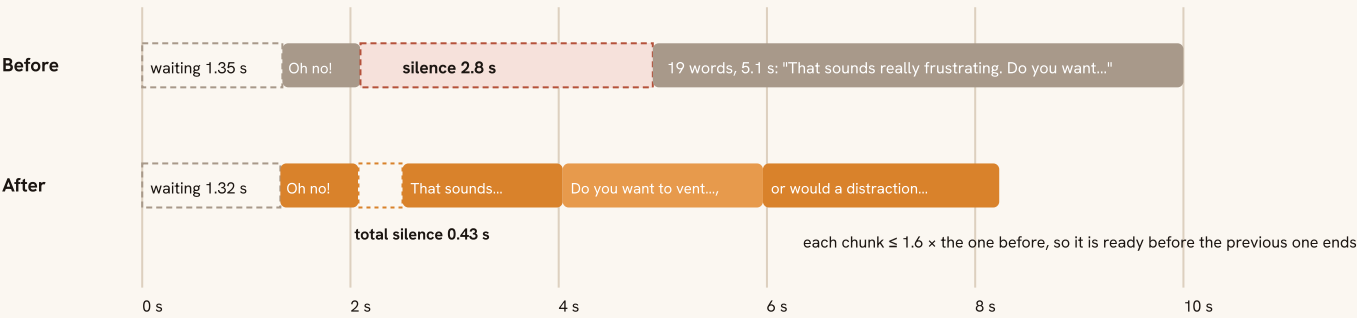


Figure 2. Benchmark reply "ohNo" at 0.65× real-time inference. Measured: first audio 1.35 s → 1.32 s, silence 2.80 s → 0.43 s. "Before" bars use measured lengths; the lengths of the later "after" chunks are estimated from their word counts.

The rule

- First chunk:** the first sentence goes out at once. Only a one-word opener ("Yes!") waits for the next sentence. An opening sentence over 80 characters is cut at its last clause boundary, even while it is still streaming, but only where the Markdown is balanced.
- Later chunks:** whole sentences, up to `max(60, 1.6 × previous)` characters. A sentence is split at a clause only when it alone exceeds that budget. Hard cap 280.
- Never splits inside a code fence or link, and never before an abbreviation (see 08).

Why these numbers

They come from a sweep over six replies with real Kokoro audio lengths, at 0.4× and 0.62× inference. At 0.62×:

OPTION	TTFA	SILENCE
Old rule	2676	2096
Chosen	1774	438
First chunk ≥ 5 words	2369	310
"15-word minimum" guideline	4637	3

ms, averaged per reply. The 20–60-word guideline from the brief was tested and rejected: it removes gaps but nearly doubles the wait before audio starts.

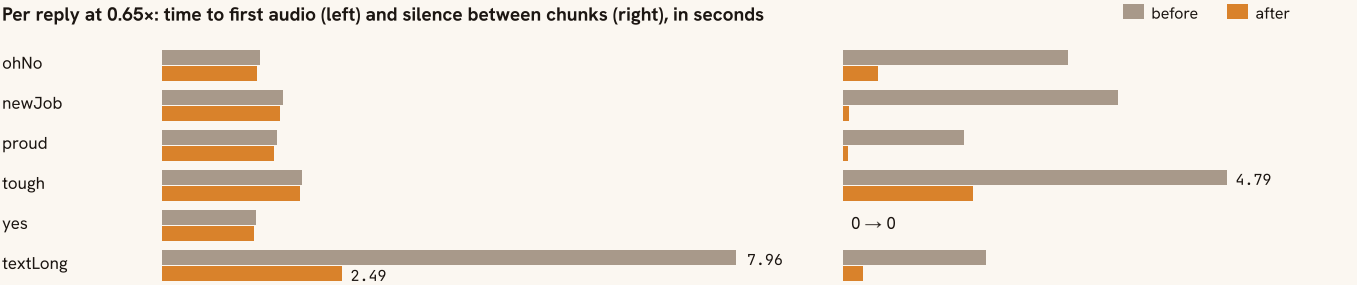


Figure 3. From `npm run bench-tts -w server` at 2 threads. First audio is never later than before on any reply, and silence drops on every reply. The trade-off is that short replies are spoken in 3–5 pieces instead of 2–3, and each piece is still a whole sentence or clause.

Gapless playback on one AudioContext NEW

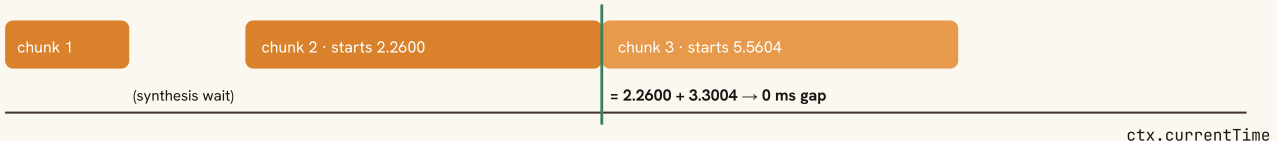
Before, every chunk was a new `<audio>` element that could start only after the previous element's `ended` event. Every chunk also built a brand-new AudioContext just to drive the orb's level meter, and routed the element through it. A context created without a user gesture can start *suspended*, and an element routed through a suspended context plays silence.

Before: event-driven hand-off



If "ended" never fires (element stalled, blocked or muted), the queue waited forever. No watchdog.

After: one clock, one cursor



Each decoded buffer is scheduled with `start(max(now + 20 ms, scheduledUntil))`. The pump hands control back 250 ms before the end, so the next, already-decoded chunk lands on the exact end sample.

Figure 4. Numbers measured in headless Chromium against the live server. The context ran at 48 kHz and the 24 kHz WAVs decoded to their exact lengths (1.146 s vs. the server's 1146 ms). The first gap is synthesis time, not playback.

What the module does

- One shared `AudioContext` and `AnalyserNode` (the orb level), created on first use.
- **Unlock:** on any pointer, key or touch it calls `resume()` and plays one silent sample. That is what iOS and Safari need before later, non-gesture audio may play.
- `ensureRunning()` before each chunk. If the context is still locked, that chunk plays through an `<audio>` element instead (the old path).
- `stopAll()` stops every scheduled source immediately and wakes any waiter.
- **Stall watchdog:** if the audio clock doesn't move for 4 s (for example iOS interrupted it), waiting callers are released instead of freezing.

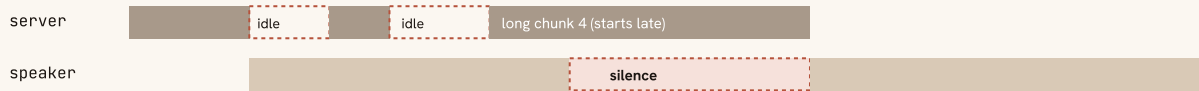
Why it is better

- No per-chunk start-up cost and no dependence on an event to begin the next chunk. Gapless playback by construction [Smus ch. 2].
- One context instead of one per chunk. Nothing is routed through a context that may be suspended.
- Follows the autoplay rules: Chrome starts contexts created before a gesture as suspended and resumes them after one; iOS needs a single gesture unlock.
- Verified in real Chromium: scheduling exact to 4 decimal places, and Stop ends the playing chunk at once with no later chunk starting.

Prefetch depth 2 (was 1) CHANGED

The server handles one inference at a time. With depth 1, chunk N+2 was requested only once chunk N+1 began playing. If N+1 was short, the server sat idle while it played, and a longer chunk later arrived late. Depth 2 keeps one extra request queued so the server builds up a lead.

Depth 1: the server waits for playback



Depth 2: the server stays busy and banks time



Figure 5. Illustrative. Measured effect with the *old* chunks: silence per reply 2.38 s → 1.98 s, and on the long reply 1.78 s → 0.02 s (bench strategies "legacy" vs. "legacyChunksNewPlayback").

Why not deeper? Every extra chunk requested is synthesis a Stop throws away. Chunks beyond 2 add nothing when the server is already continuously busy. Backpressure is enforced at both ends: at most 2 look-ahead requests per listener, and a server queue of 8.

The first chunk of a reply is still requested alone. Its answer decides the voice engine (neural or browser) once, so the "one voice per reply" rule stays race-free. `verify-voice.mts` still passes in all three of its failure modes against the live server.

Cancellation, timeouts, retries, watchdogs FIX

The main cause of "speech gets stuck": `fetch('/api/tts')` had no timeout and no signal. One request that never answered held the queue's `pumping` flag forever, so every later reply queued behind it in silence. Stop and new messages discarded the audio when it arrived, but never cancelled the request, so the server kept synthesizing chunks nobody would hear.

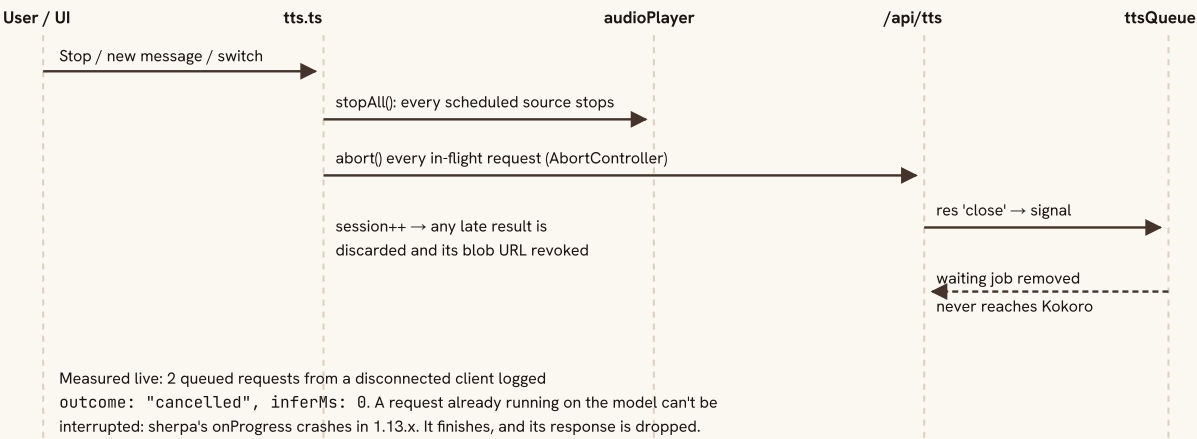


Figure 6. What happens on Stop. Tests cover Stop while speaking, a new message while old audio is still being generated, and 4 rapid consecutive replies (only the last is audible and the first 3 requests are aborted).

GUARD	VALUE	WHY
Request timeout	35 s	Above the server's own limits (10 s wait + 20 s inference), so the server normally answers first. It is a hard bound on "stuck".
Retry	once	Only for a real network failure (<code>TypeError</code>) or a <code>503</code> carrying <code>Retry-After</code> (server busy). Never after abort, timeout, a plain 503 (model unavailable) or 4xx.
<audio> watchdog	length + 5 s	The length is read from the WAV header, so an element that never fires <code>ended</code> can't freeze the reply.
Browser voice	≤ 200 chars + watchdog	Chrome's <code>speechSynthesis</code> stops about 15 s into an utterance without firing <code>onend</code> (Chromium issue 41294170). Utterances used to go up to 320 characters (about 20 s).
Correlation	X-TTS-Generation / -Chunk	Numbers only, so server logs can be joined per reply without any text.

Bounded server queue and backpressure

NEW

Before, the server used a promise-chain mutex. It kept synthesis serial, but it had no depth limit, no wait limit and no inference limit, and it checked for cancellation only before a request joined the chain. Under load, every request waited in line, however stale.

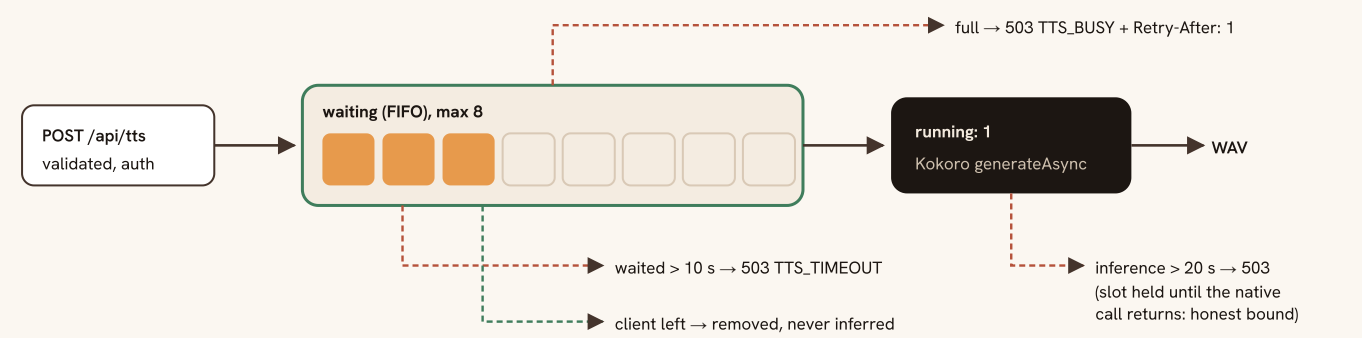


Figure 7. All limits are configurable: TTS_CONCURRENCY, TTS_MAX_QUEUE, TTS_QUEUE_TIMEOUT_MS, TTS_INFERENCE_TIMEOUT_MS.

Live burst: 12 concurrent requests

RESULT	COUNT
200 audio/wav	7
503 TTS_BUSY, Retry-After: 1 (immediate)	3
503 TTS_TIMEOUT (after 10 s)	2

The backlog stays bounded, and nothing waits minutes on a socket. The client retries a busy 503 once, then skips the chunk without switching voices.

Why concurrency 1

Two parallel inferences on one model instance finished together in 3.4 s, against 3.1 s for one alone. They split the same CPU cores, so nothing arrives sooner. Throughput on hosts with spare cores comes from **threads per inference** (07). The espeak front end's thread-safety is also undocumented, so one at a time is the safe default.

Measured event-loop lag during inference was ≤ 27 ms, so `generateAsync` already runs off the main thread. **No worker thread is needed**, and none was added.

Threads, warm-up, health and logs

CHANGED

Real-time factor by threads (lower is faster)

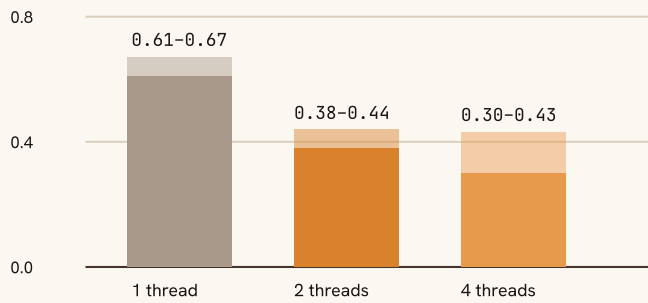


Figure 8. Idle machine, short/medium/long chunks. Solid = best case, faded = range.

TTS_NUM_THREADS=auto

Threads were hard-coded to 1. `auto` reads the container's CPU grant (cgroup v2 `cpu.max`, falling back to v1) and caps it at 2. On Linux, when the grant can't be read, it uses 1. That matters because ONNX Runtime otherwise sizes its pool from the *host's* cores, which oversubscribes a small CPU share (ORT threading docs).

2 threads give -37% inference time; 4 add little for short chunks and take cores from the event loop and other users. A fractional-CPU host (Render free: 0.1) stays at 1, exactly as before.

CHANGE	WHY	EVIDENCE
Warm-up: one "Hi." through the queue right after load	The first real reply doesn't pay first-run costs (espeak data, ORT allocations). It is queued first, so a real request waits at most one short job.	50-130 ms on the M5 (within noise under load). Kept because it is almost free; not claimed as a big win.
<code>GET /api/tts/health</code> (public, no content)	Load and warm state, threads, queue depth, counters (ok, cancelled, busy, timeouts, failures), last RTF. A <code>lastRtf</code> near 1 means the host is too slow for gapless speech.	Route test; live output checked
One JSON log line per chunk	<code>reqId, generation, chunk, model, voice, textLen, queueWaitMs, inferMs, audioMs, rtf, outcome, errorCategory</code> . Never the text. Jobs that time out in the queue report how long they waited.	A search of the live log for any request text found nothing
Emoji-only input → 50 ms of silence	An error here would make the client lock the reply onto the browser voice	Unit test

Not changed: the runtime and model. Kokoro v1.0 fp32 on ONNX through sherpa-onnx stays. PyTorch doesn't apply (Node, no GPU). int8 was rejected earlier on quality grounds. CoreML is macOS-only and production is Linux. Memory remains the big hosting issue: about 600 MB more RSS against Render free's 512 MB (see 11).

Speakable text: emoji, URLs, abbreviations, prompt FIX

Every rule here came from measuring the real model: the same sentence was synthesized raw and normalized, and the audio lengths compared. A much longer raw version means the voice is reading out something a person wouldn't say. Rules were added only where that happened.

Audio length, raw vs. normalized (seconds). Scale: 1 s = 60 px

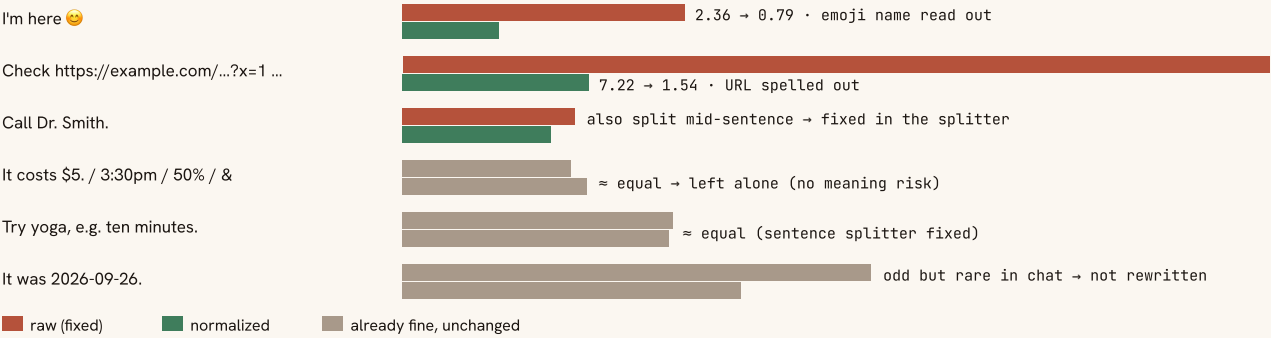


Figure 9. Raw vs. normalized audio length for the same sentence.

RULE	WHERE	DETAIL
Drop emoji	client + server	Pictographs plus the joiners, variation selectors, skin tones and flags that build them. Digits, # and * are kept, so "👉" still says "1".
Bare URL → site name	client + server	"https://www.example.com/a?b" → "example.com" (the voice says "example dot com"). Trailing punctuation is kept. Markdown link labels were already handled.
No sentence end at abbreviations	client	Mr, Mrs, Ms, Dr, Prof, Sr, Jr, vs, e.g., i.e. "etc." ends a sentence only before a capital letter. Avoids a request boundary (and an intonation reset) in mid-sentence.
Voice-mode prompt line	server	"Use contractions and everyday words. No emojis, links, or code: name a website instead of spelling out its address...". Added to the VOICE MODE layer only, so text chat is untouched (test-pinned).
Keep punctuation	client	Clause cuts keep their comma. No commas or ellipses are ever inserted, because punctuation drives Kokoro's pauses.

Voice and speed unchanged: af_heart is Kokoro's only grade-A American female voice (af_bella is A-, per VOICES.md). Speed 0.95 is inside the suggested 0.92–1.02 and already configurable and clamped. Neither was A/B'd by ear.

Languages Kokoro can't speak NEW

An honest limit: text normalization cannot rescue a weak language model. Kokoro v1.0's Hindi voices are grade C, it has no Gujarati at all, and the app sent every chunk, whatever its script, to the English voice.

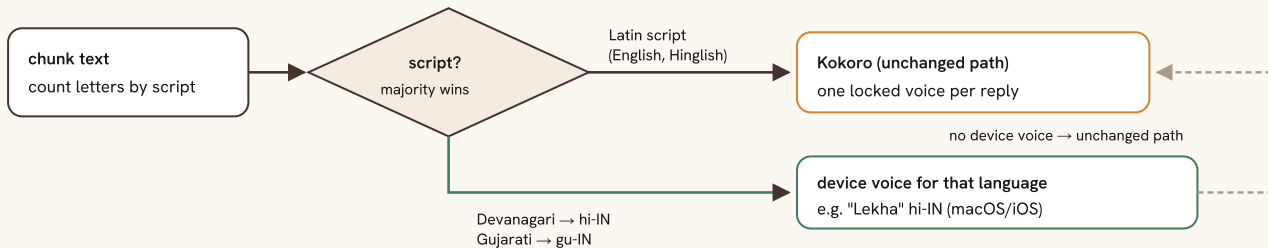


Figure 10. A clean provider seam. A future Hindi or Gujarati engine would slot in where the device voice sits, without touching conversation logic.

- This is the one deliberate exception to "one voice per reply": a different voice beats an unintelligible one. Pure English replies are completely unaffected (tests pin both cases).
- **Hinglish** (Hindi written in Latin script) can't be told apart from English, so it stays on the English voice with English phonetics. That is a known limitation.
- Better engines were considered and **not** added: Piper hi-IN-priyamvada (female, 21 MB, same API) is **CC-BY-NC-SA 4.0, non-commercial**. A second Kokoro instance for Hindi costs about 600 MB more RAM for grade-C voices. AI4Bharat Indic Parler-TTS covers Gujarati but needs a GPU-class Python service. Sarvam Bulbul is a **paid API with credentials**. See the audit, §9.

Speech-to-text: the Brave fallback could never succeed FIX

When Brave's recognizer fails with `network`, the recorded clip is uploaded for server transcription. Commit dc671a0 raised the turn cap from 8 s to 20 s for browser recognition, and the fallback kept recording for that whole cap. The server rejects any clip over `TRANSCRIPTION_MAX_DURATION_MS` (12 s) with **413**. Every Brave fallback turn therefore failed, and each one first made the user wait the full window.

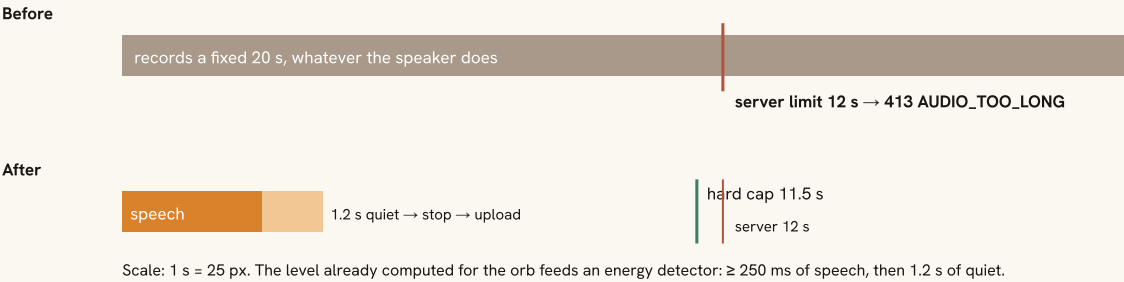


Figure 11. The 1.2 s pause matches `SILENCE_COMMIT_MS` on the browser-recognizer path. Blips shorter than 250 ms (clicks) are ignored, and a turn never ends before speech was heard.

Tests: the old code would produce a clip of about 20 000 ms (asserted ≤ 12 000 now). The detector ends after a pause, waits through a 600 ms breath, ignores a 100 ms click, and never ends on silence alone.

Other speech-to-text findings: documented, not changed

FINDING	RECOMMENDATION
Each turn opens <code>getUserMedia</code> and a <code>MediaRecorder</code> even when browser recognition works: two mic captures and Opus encoding for nothing. On iOS, the open mic switches the audio session to play-and-record, which can send TTS to the earpiece, and the barge-in probe does this while TTS plays.	Record only when the fallback is possible, or release the mic before speaking on iOS. Needs device testing.
Barge-in can trigger on the persona's own voice through speakers. SpeechRecognition captures separately, so echo cancellation doesn't apply to it.	Require about 2 words or 300 ms of interim speech, and ignore interim text that matches the words currently being spoken.
Fallback is paid (OpenAI whisper-1), and the language is fixed to English.	Chrome's on-device recognition (<code>processLocally</code> , Chrome 139+), or offline ASR in the sherpa-onnx addon already installed. Both are free and private.

Files, tests, validation, rollback, limitations

Files

FILE	ROLE
client/.../speechChunker.ts	Chunk sizing (02) NEW
client/.../audioPlayer.ts	Web Audio engine (03) NEW
client/.../tts.ts	TTS queue, moved out of speech.ts (04, 05) NEW
client/.../browserVoice.ts	Fallback and language voices (05, 09) NEW
client/.../speech.ts	STT only, re-exports the TTS API
client/.../speechText.ts	Emoji, URLs, abbreviations, script detection
client/.../voiceCapture.ts	Brave fallback fix (10)
client/.../chatStore.ts	Uses SpeechChunker
server/.../ttsQueue.ts	Bounded queue (06) NEW
server/.../ttsText.ts	Server-side emoji and URL guard NEW
server/.../ttsService.ts	Queue, timeouts, threads, warm-up, logs
server/.../routes/tts.ts	Error codes, Retry-After, health
server/.../config/index.ts	6 optional settings
server/.../models/Persona.ts	Voice-mode prompt line
server/.../scripts/bench-tts.mts	Benchmark NEW
docs/deployment.md, AGENTS.md	Settings, health, RAM, agent notes

Rollback

There is no model or runtime change, so a **git revert** of the commit restores the old behaviour exactly. Without a deploy: TTS_NUM_THREADS=1 gives the old threading, TTS_WARMUP=false skips warm-up, and very large TTS_MAX_QUEUE and TTS_QUEUE_TIMEOUT_MS values approximate the old unbounded queue.

Validation

- **Tests:** server 79 → 103, client 228 → 275, all passing. New coverage: chunk order and sizes, no lost words, no Markdown leaks, queue limits and FIFO, cancellation at dequeue, busy and timeout mapping, disconnect cleanup, stale-generation rejection, Stop while speaking, a new message during generation, rapid consecutive replies, many concurrent users (20 jobs, peak concurrency held at 2), timeout fallback, retry rules, both watchdogs, emoji, Markdown, mixed-language and empty text.
- **Lint and build:** clean. The local node_modules lacked the locked ESLint plugins; npm install fixed that and left the lockfile unchanged.
- **Live server:** binary WAV at 24 kHz; burst and disconnect behaviour as shown in 06; verify-voice.mts PASS x3.
- **Real Chromium:** gapless scheduling and immediate Stop (03).
- **Benchmark:** repeatable. Before and after run in the same process under the same load.

Configuration you may want

- Nothing is required. Optional settings: TTS_NUM_THREADS, TTS_CONCURRENCY, TTS_MAX_QUEUE, TTS_QUEUE_TIMEOUT_MS, TTS_INFERENCE_TIMEOUT_MS, TTS_WARMUP.
- For neural TTS in production: ≥ 1.5 GB RAM (the model adds about 600 MB of RSS, more than Render free's 512 MB) and ≥ 2 dedicated vCPUs for the thread speed-up.

Still open

- Inference already running can't be interrupted (the onProgress crash). Re-test on sherpa-onnx-node 1.13.8.
- If the host is slower than real time (> 1x), gaps remain. That is a hardware limit.
- Before the first user gesture, Web Audio may be locked; playback then uses the old element path.
- Speech-to-text: double mic capture, barge-in echo, paid English-only fallback.
- Voice quality was not judged by ear.